

Relatório Final - Projeto AquaSense

Universidade: IADE - Universidade Europeia

Unidade Curricular: PBL para 4 UC's (IoT, Sistemas Distribuídos, Engenharia de Software e IA)

Grupo e Ano Letivo: Grupo 3 (2025/2026)

Autores: Rui Outeiro (20231566), Emanuel Carvalho (20231627), Paulo Jadaugy (20241711)

Índice

- Relatório Final - Projeto AquaSense
 - Índice
 - Introdução
 - Público-Alvo
 - Objetivo do Projeto
 - Identificação do Problema
 - Solução Proposta
 - Identificação de Requisitos
 - Estruturação da arquitetura
 - Arquitetura por Camadas IoT
 - Ferramentas de Desenvolvimento
 - Infraestrutura Computacional
 - Comunicação entre Módulos
 - Esboço do Artefacto Físico
 - Descrição da Solução e Arquitetura Implementada
 - Desenvolvimento e Prototipagem
 - Descrição de Atividades Realizadas
 - Integração de IA, Interação Natural e Sistema Completo
 - Testes e Resultados
 - Plano de Trabalho e Distribuição de Tarefas
 - Fases do Projeto
 - Enquadramento nas Unidades Curriculares
 - Próximas Etapas
 - Conclusão
-

Introdução

O projeto **AquaSense** surge como resposta à necessidade de modernizar e automatizar a gestão de aquários domésticos. Combinando tecnologias de Internet das Coisas (IoT), Inteligência Artificial e desenvolvimento web, o sistema oferece uma solução completa para monitorização e controlo remoto de parâmetros aquáticos essenciais.

Este relatório documenta o desenvolvimento integral do projeto, desde a conceptualização até à implementação funcional, abrangendo todas as componentes de hardware, software e integração.

Público-Alvo

O AquaSense destina-se a:

- **Aquarofilistas amadores** - Entusiastas que mantêm aquários em casa e procuram uma forma simples de monitorizar os seus sistemas sem necessidade de conhecimentos técnicos avançados.
- **Aquarofilistas experientes** - Utilizadores com múltiplos aquários que necessitam de uma solução centralizada para gerir e automatizar a manutenção.
- **Criadores de peixes ornamentais** - Profissionais que precisam de monitorização contínua e alertas em tempo real para proteger os seus investimentos.
- **Instituições educacionais** - Escolas e universidades com aquários didáticos que beneficiam de dados históricos e análise automatizada.
- **Lojas de aquarofilia** - Estabelecimentos comerciais que necessitam de monitorizar múltiplos sistemas de exposição.

Perfil do utilizador típico:

- Idade: 18-65 anos
 - Conhecimentos técnicos: Básicos a intermédios
 - Dispositivos: Smartphone ou computador com acesso à internet
 - Motivação: Reduzir tempo de manutenção, prevenir perdas e otimizar condições do aquário
-

Objetivo do Projeto

Objetivo Geral

Desenvolver um sistema inteligente de monitorização e controlo de aquários que permita aos utilizadores acompanhar em tempo real os parâmetros da água, automatizar processos de manutenção e receber recomendações baseadas em inteligência artificial.

Objetivos Específicos

1. **Monitorização contínua** - Implementar sensores para recolha de dados de temperatura, pH, turbidez e condições ambientais.
2. **Automação inteligente** - Criar sistemas de controlo automático para iluminação e ventilação baseados em regras e aprendizagem.
3. **Interface acessível** - Desenvolver uma aplicação web responsiva e intuitiva para visualização e configuração.
4. **Alertas proativos** - Implementar sistema de notificações via Telegram para situações críticas.

5. **Análise preditiva** - Integrar modelos de IA para sugestões de otimização do fotoperíodo e detecção de anomalias.
6. **Custo acessível** - Criar uma solução economicamente viável comparada às alternativas comerciais existentes.

Identificação do Problema

Uma vez que não existe no mercado um sistema completo que permita acompanhar a qualidade da água em tempo real remotamente, nomeadamente parâmetros como pH, turbidez e temperatura e condições ambientais ao redor do aquário como temperatura e humidade.

Além disso, na aquariofilia, é preciso disciplina na manutenção dos aquários, pelo que podem existir momentos de desleixe porque requer muito trabalho. O objectivo aqui é automatizar parte desse trabalho como a monitorização de alguns dos parâmetros, a iluminação automática, etc.

As soluções atualmente disponíveis no mercado são limitadas, e não têm a capacidade de análise via Inteligência Artificial dos dados recolhidos para decisões operacionais. Além disso as poucas e limitadas ferramentas que existem são extremamente caras.

Devido a estas situações torna-se relevante desenvolver uma solução inteligente, acessível e inovadora que seja capaz de recolher dados, processar e analisar os dados da água em tempo real e que recorre a inteligência artificial para tomar decisões e emitir alertas para ajudar na manutenção dos aquários.

Solução Proposta

O sistema Aquasense permite a monitorização contínua e automática da qualidade da água do aquário, ao recolher dados de pH, turbidez e temperatura, estes são dados incompletos para uma análise completa da qualidade da água, mas permite ter uma ideia clara de alguns parâmetros chave.

O sistema é capaz de processar e analisar os dados em tempo real, permitindo atuação automática e emissão de alertas em casos de condições críticas.

Permite acesso remoto à informação e permite a configuração do fotoperíodo e ventilação pelo utilizador e também recorre a inteligência artificial para apoiar a tomada de decisão e otimizar a manutenção do aquário.

Identificação de Requisitos

Foi elaborado um relatório com todos os requisitos funcionais/não funcionais que o sistema deve respeitar.

Desse documento destacamos as seguintes funcionalidades:

- **Sistema de iluminação inteligente**
 - Simulação de nascer e pôr do sol através de dimming (PWM), com fotoperíodo definido pelo utilizador na aplicação móvel.
 - Ajuste automático do fotoperíodo através de inteligência artificial, com base na claridade da água medida por um LDR que analisa a quantidade de luz que atravessa a coluna de água, permitindo detectar indiretamente a presença de algas e ajustar o fotoperíodo em conformidade.
 - Integração de uma API de meteorologia para adaptar o fotoperíodo às condições exteriores (ex: dias muito escuros, trovoadas, vagas de calor), aproximando o ciclo de luz no aquário das

condições naturais. [pode ser desactivado pelo utilizador]

- **Conectividade e aplicação móvel**

- Comunicação via wifi entre o ESP32 e o backend.
- Aplicação móvel para:
 - Configuração do fotoperíodo.
 - Consultas em tempo real, e históricos de medições.
 - Receção de alertas e notificações com sugestões de acções a tomar perante as situações.

- **Monitorização contínua de parâmetros**

- PH da água.
- Temperatura da água.
- Temperatura ambiente.

- **Alertas**

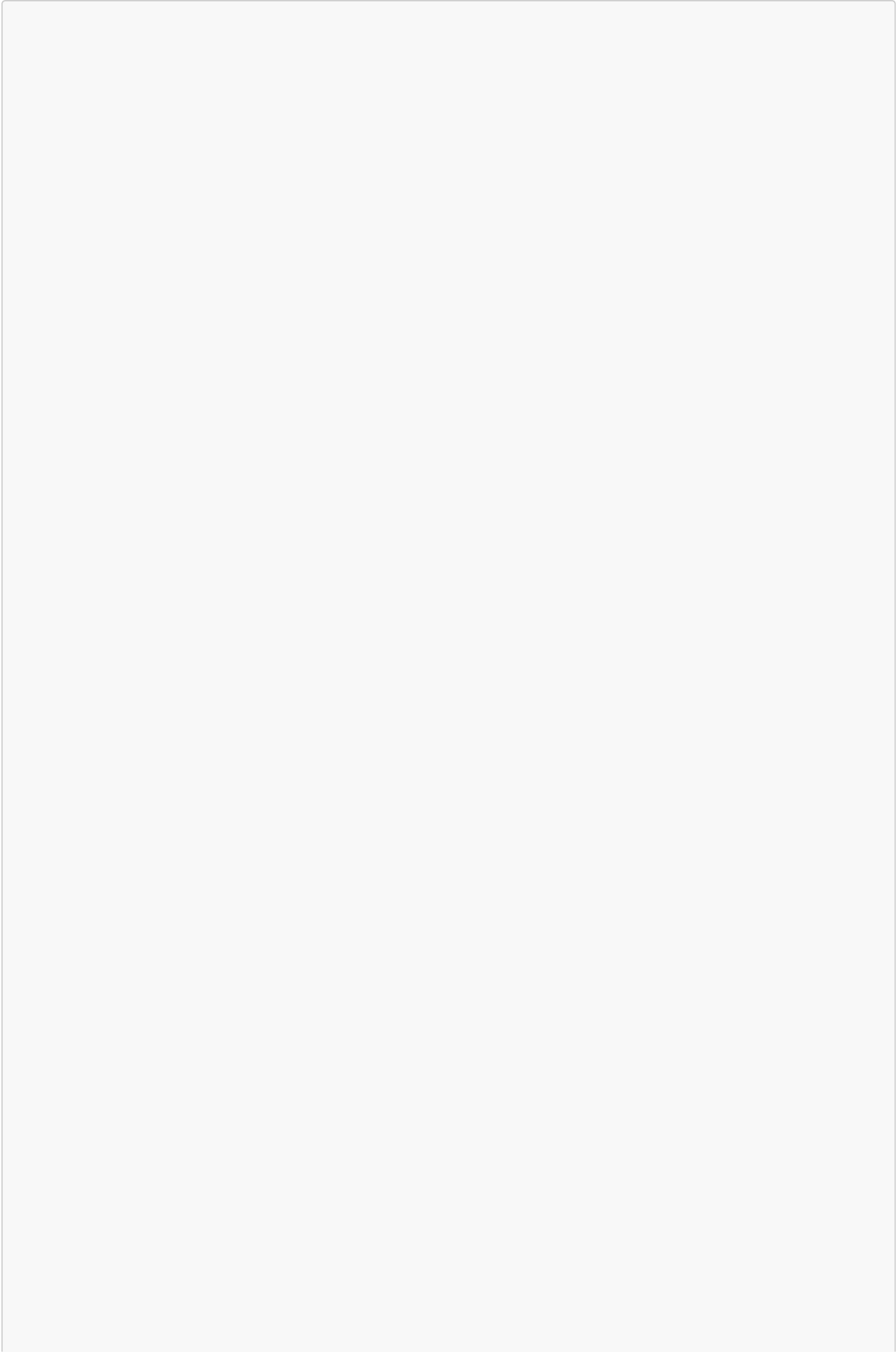
- Notificações quando qualquer parâmetro sai dos intervalos definidos como seguros (PH, temperatura da água, temperatura ambiente, claridade anormal da água).

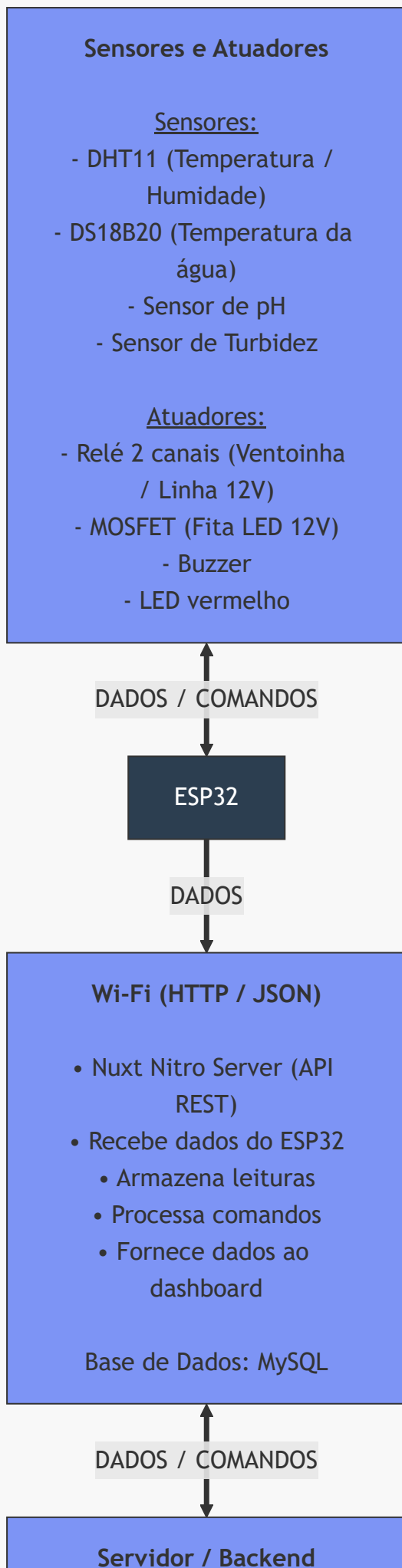
- **Arrefecimento automático**

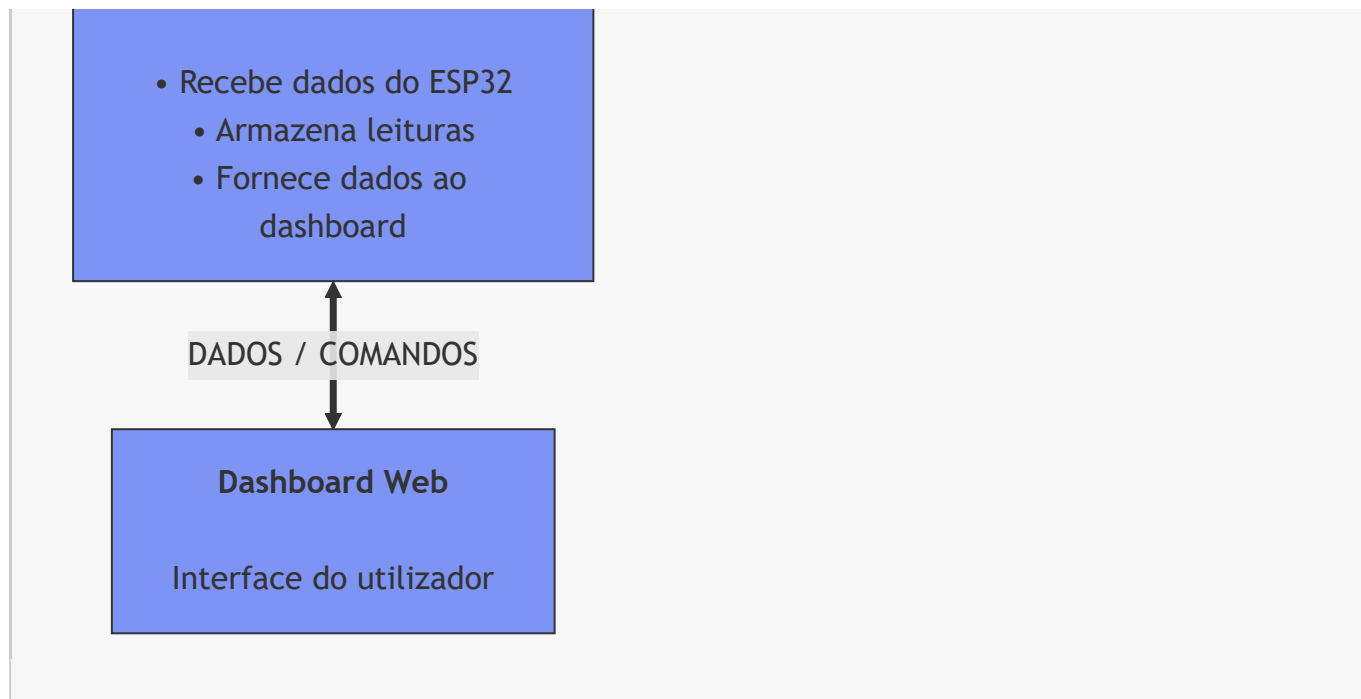
- Acionamento de uma ventoinha de arrefecimento quando a temperatura da água atinge ou ultrapassa os 30 °C, ajudando a manter o aquário dentro de uma faixa térmica segura.

[Link para o relatório](#)

Estruturação da arquitetura





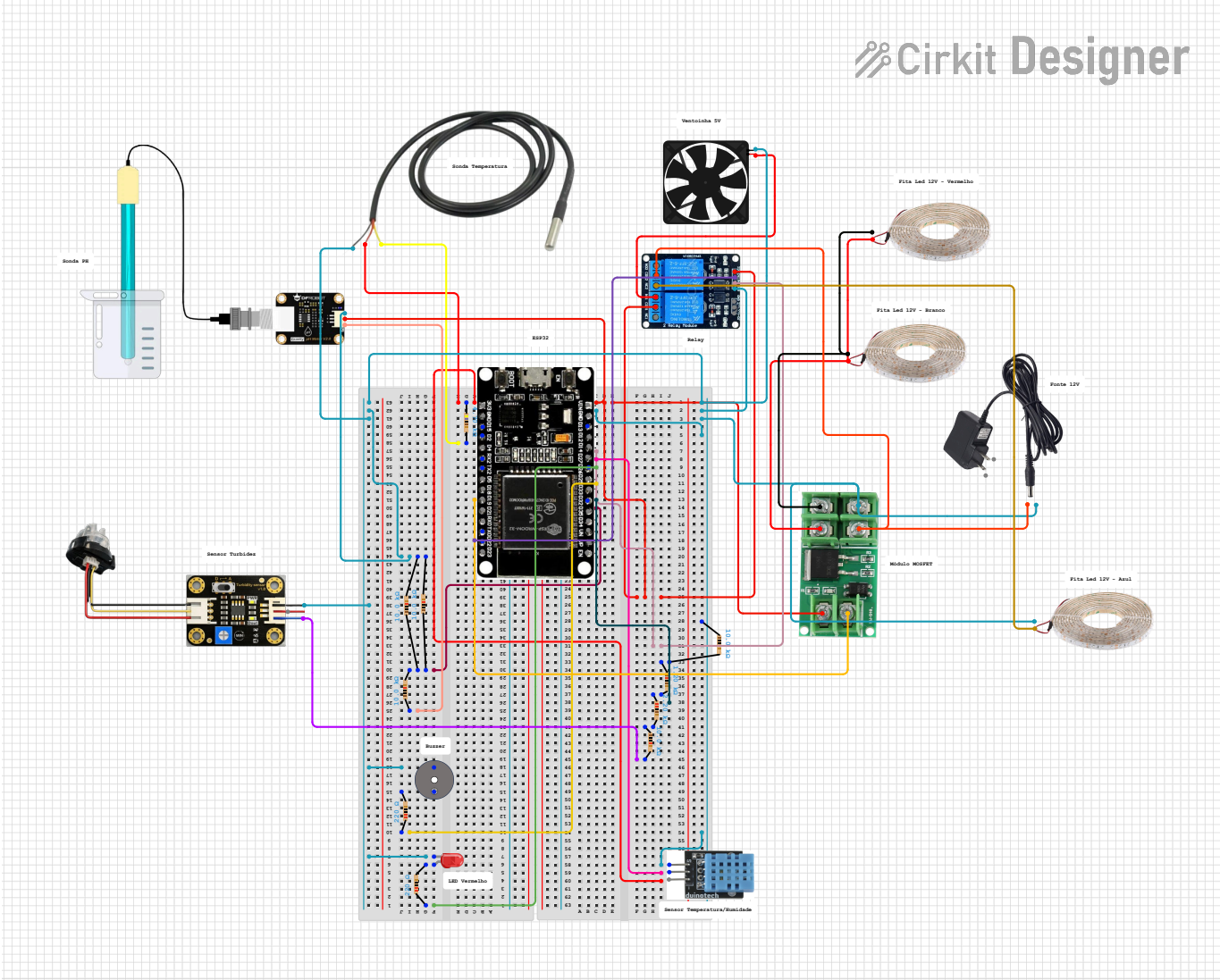


Arquitetura por Camadas IoT

1. Camada de Percepção (Dispositivos)

Responsável pela recolha de dados do ambiente físico e atuação sobre o aquário.

Hardware



[Link do projeto no Cirkít Designer](#)

Ligações ao ESP32:

GPIO	Componente	Tipo	Função
GPIO 4	DS18B20	OneWire	Temperatura da água
GPIO 26	DHT11	Digital	Temperatura/humidade ambiente
GPIO 34	Sensor pH	ADC	Leitura de pH
GPIO 35	Sensor turbidez	ADC	Leitura de turbidez
GPIO 27	Relé K1	Digital	Ventoinha (ativo LOW)
GPIO 25	LED vermelho	Digital	Indicador ventoinha
GPIO 33	Buzzer passivo	PWM	Alerta sonoro
GPIO 21	MOSFET	PWM	Luz principal 12V
GPIO 23	Relé K2	Digital	Luz noturna

Material:

Componente	Qtd	Notas
ESP32	1	Controlador principal
DS18B20	1	Temp. água
DHT11	1	Temp/humidade ambiente
Sensor pH + módulo	1	Leitura pH
Sensor turbidez	1	Leitura turbidez
Relé 2 canais	1	Ventoinha + luz noturna
MOSFET (IRLZ44N)	1	PWM LED 12V
Ventoinha 5V	1	Arrefecimento
Fita LED branca 12V	5m	Iluminação principal
Fita LED azul 12V	3m	Iluminação noturna
Fonte 12V + 5V	2	Alimentação

Software (Firmware ESP32)

Biblioteca	Função
WiFi.h	Conectividade WiFi
HTTPClient.h	Comunicação REST
ArduinoJson	Parsing JSON
OneWire + DallasTemperature	Sensor DS18B20
DHT	Sensor DHT11
time.h	Sincronização NTP
LEDC	Controlo PWM

Processamento

- Leitura periódica dos sensores (temperatura, pH, turbidez, humidade)
- Controlo automático da ventoinha por temperatura
- Controlo de iluminação por horário/modo com fade PWM
- Envio de dados em batch via HTTP POST (JSON)

Segurança

- Validação de leituras antes do envio
- Reconexão automática WiFi
- Timeout em requests HTTP

2. Camada de Rede (Conectividade)

Responsável pela comunicação entre dispositivos e servidor.

Protocolo

- **WiFi 2.4GHz** - Ligação do ESP32 à rede local
- **HTTP/REST** - Comunicação ESP32 ↔ Backend
- **JSON** - Formato de dados

Fluxo de Dados

ESP32 → HTTP POST /api/sensors → Nitro Backend → MySQL
ESP32 ← HTTP GET /api/config/esp32 ← Nitro Backend ← MySQL

Endpoints Principais

Método	Endpoint	Função
POST	/api/sensors	Receber dados dos sensores
GET	/api/config/esp32	Enviar configurações ao ESP32
GET/PUT	/api/config	Configurações do dashboard
GET/PUT	/api/alertas/config	Configuração de alertas
POST	/api/auth/login	Autenticação

Segurança

- JWT para autenticação de utilizadores
- Cookies HTTP-only para sessões
- CORS configurado

3. Camada de Processamento (Backend + IA)

Responsável pelo armazenamento, processamento e análise inteligente dos dados.

Backend (Nitro/Nuxt)

Componente	Tecnologia	Função
Runtime	Node.js	Execução
Framework	Nuxt 4 / Nitro	API REST
Base de Dados	MySQL	Persistência

Componente	Tecnologia	Função
Auth	JWT + bcrypt	Autenticação

Inteligência Artificial

Componente	Tecnologia	Função
Rede Neural	PyTorch	Modelo preditivo
API Server	Flask	Exposição REST
Preprocessing	scikit-learn	Normalização dados

Endpoints IA:

Endpoint	Função
GET /api/ai/health	Status do modelo
GET /api/ai/photoperiod	Sugestão de fotoperíodo
POST /api/ai/apply	Aplicar sugestão

Processamento

- Armazenamento de leituras históricas
- Cálculo de médias e estatísticas
- Verificação de limites e geração de alertas
- Inferência do modelo neural para sugestões

Segurança

- Passwords com hash bcrypt (salt 10)
- Tokens JWT com expiração
- Validação de inputs em todos os endpoints

4. Camada de Aplicação (Frontend)

Responsável pela interface com o utilizador.

Tecnologias

Componente	Tecnologia
Framework	Nuxt 4 + Vue 3
Styling	CSS custom
Icons	Material Icons
Fonts	Inter, JetBrains Mono

Funcionalidades

- Dashboard em tempo real com gráficos
- Configuração de fotoperíodo e ventilação
- Gestão de alertas e notificações
- Histórico de leituras
- Sugestões de IA
- Perfil de utilizador

Integrações

Serviço	Função
Telegram Bot API	Notificações de alertas

Segurança

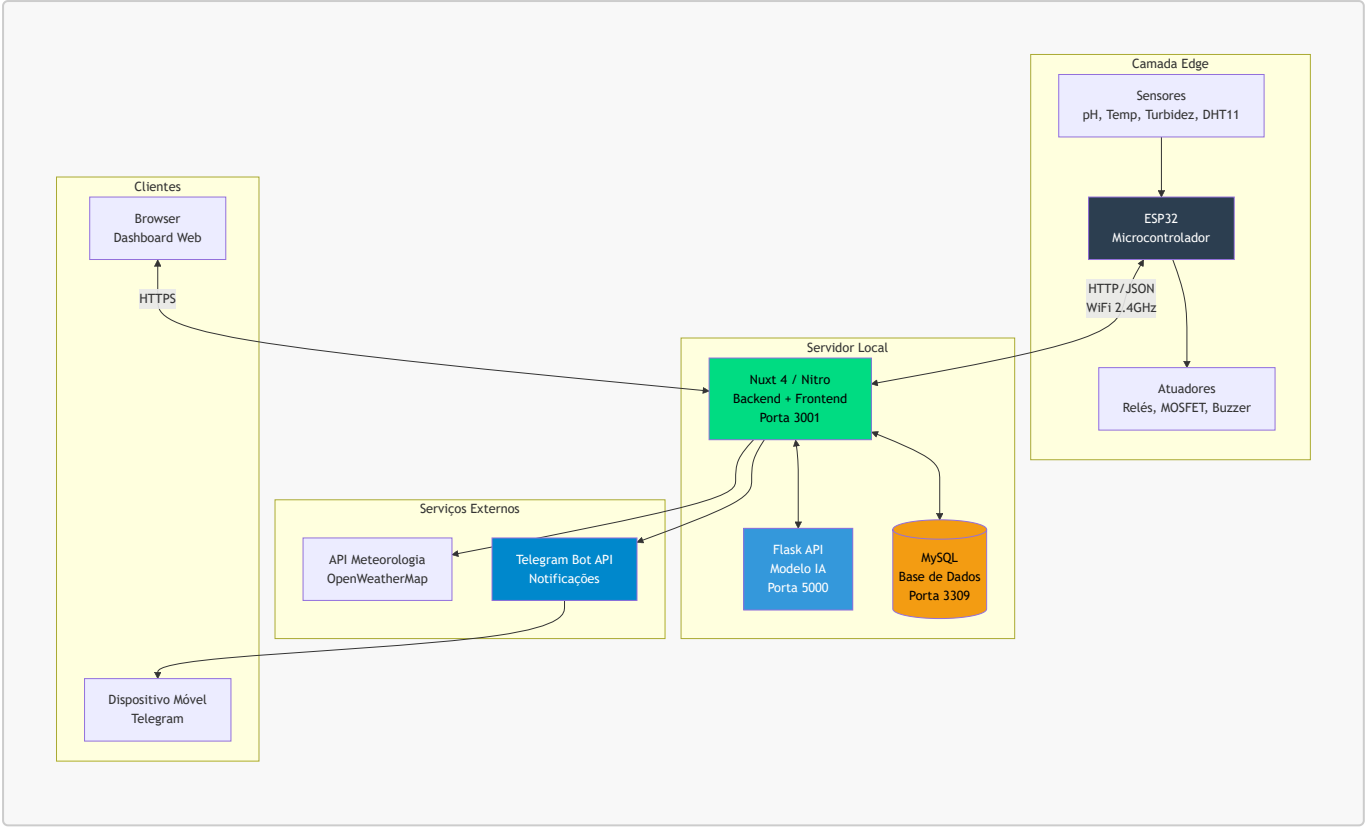
- Autenticação obrigatória
- Sessões com cookies seguros
- Logout automático

Ferramentas de Desenvolvimento

Ferramenta	Uso
VS Code	IDE principal
Arduino IDE	Firmware ESP32
Node.js	Runtime web
GitHub	Controlo de versões
XAMPP	MySQL local
Draw.io	Diagramas
Circuit Designer	Esquemas elétricos

Infraestrutura Computacional

O sistema AquaSense opera numa infraestrutura distribuída que combina dispositivos embebidos, servidores locais e serviços cloud.



Especificações dos Servidores

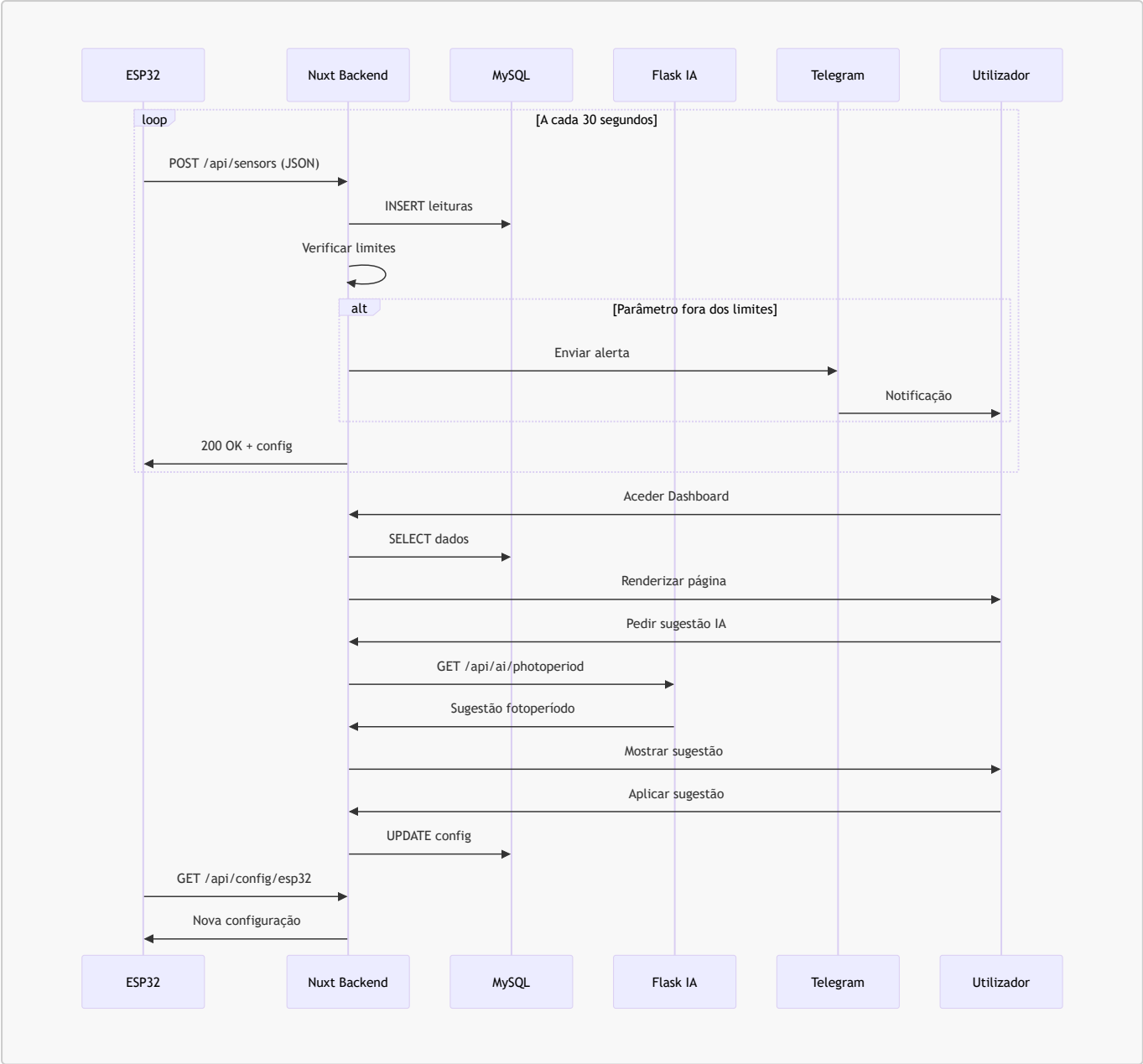
Componente	Tecnologia	Porta	Função
Backend/Frontend	Nuxt 4 (Nitro)	3001	API REST + Dashboard
Base de Dados	MySQL 8.0	3309	Persistência de dados
Servidor IA	Flask + PyTorch	5000	Inferência do modelo
ESP32	Arduino/C++	-	Recolha e atuação

Requisitos de Sistema

- **Servidor:** Node.js 18+, Python 3.9+, MySQL 8.0+
- **Rede:** WiFi 2.4GHz, acesso à internet para APIs externas
- **ESP32:** 4MB Flash, WiFi integrado

Comunicação entre Módulos

Diagrama de Comunicação



Protocolos Utilizados

Comunicação	Protocolo	Formato	Frequência
ESP32 → Backend	HTTP POST	JSON	30 segundos
Backend → ESP32	HTTP GET	JSON	Por pedido
Backend → MySQL	TCP/MySQL	SQL	Por operação
Backend → Flask IA	HTTP REST	JSON	Por pedido
Backend → Telegram	HTTPS	JSON	Por alerta

Estrutura de Mensagens

ESP32 → Backend (POST /api/sensors):

```
{
  "device_id": "esp32_01",
  "sensors": [
    {"type": "temperature", "value": 25.5, "unit": "°C"},
    {"type": "ph", "value": 7.2, "unit": "pH"},
    {"type": "turbidity", "value": 15, "unit": "NTU"},
    {"type": "ambient_temp", "value": 22.0, "unit": "°C"},
    {"type": "humidity", "value": 65, "unit": "%"}
  ],
  "timestamp": "2026-02-05T15:30:00Z"
}
```

Backend → ESP32 (GET /api/config/esp32):

```
{
  "luz_estado": true,
  "luz_intensidade": 80,
  "luz_noturna_estado": false,
  "ventoinha_estado": false,
  "modo_manual": false,
  "temp_ligar": 29,
  "temp_desligar": 27
}
```

Esboço do Artefacto Físico

Descrição Geral

O artefacto físico do AquaSense consiste numa caixa de controlo impermeável que aloja todos os componentes eletrónicos, posicionada junto ao aquário. A partir desta caixa, saem cabos para os sensores submersos e para os atuadores (iluminação e ventilação).

Componentes Físicos

Caixa de Controlo:

- Caixa estanque IP65 (aproximadamente 20x15x10 cm)
- ESP32 montado em placa de prototipagem
- Módulo relé de 2 canais para controlo de cargas
- MOSFET IRLZ44N para dimming PWM da iluminação
- Fonte de alimentação 12V/5V integrada
- Buzzer para alertas sonoros locais
- LED indicador de estado

Sensores (externos à caixa):

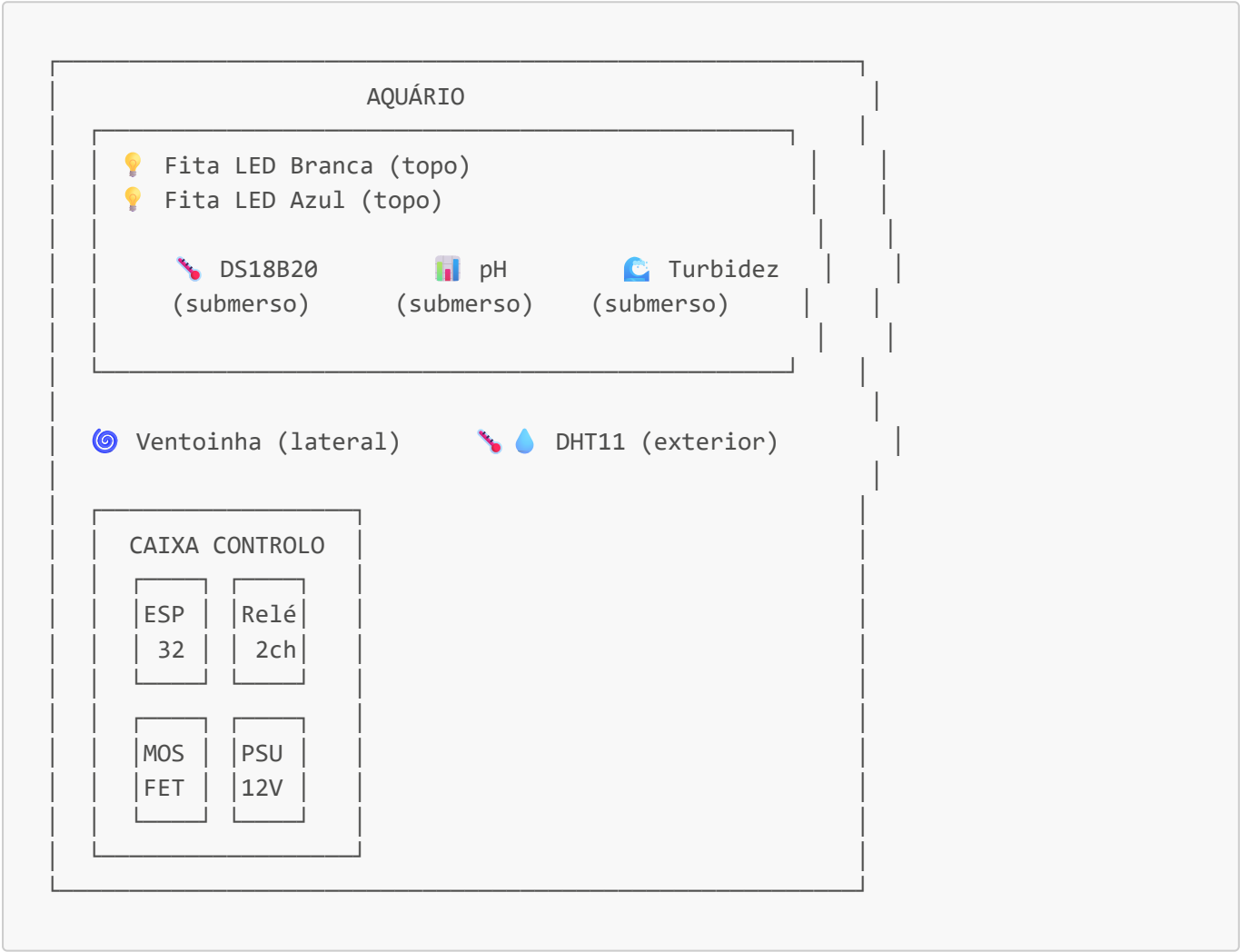
- Sonda DS18B20 à prova de água (submersa no aquário)

- Sensor de pH com sonda (submerso no aquário)
- Sensor de turbidez (submerso no aquário)
- DHT11 (exterior, mede ambiente)

Atuadores:

- Fita LED branca 12V (iluminação principal, com dimming)
- Fita LED azul 12V (iluminação noturna)
- Ventoinha 5V/12V (arrefecimento por evaporação)

Layout Físico



Fotografias do Protótipo

- [PLACEHOLDER: Foto 1]** - Vista geral do sistema montado no aquário
- [PLACEHOLDER: Foto 2]** - Interior da caixa de controlo com componentes
- [PLACEHOLDER: Foto 3]** - Detalhe dos sensores submersos
- [PLACEHOLDER: Foto 4]** - Sistema de iluminação instalado
- [PLACEHOLDER: Foto 5]** - Dashboard em funcionamento

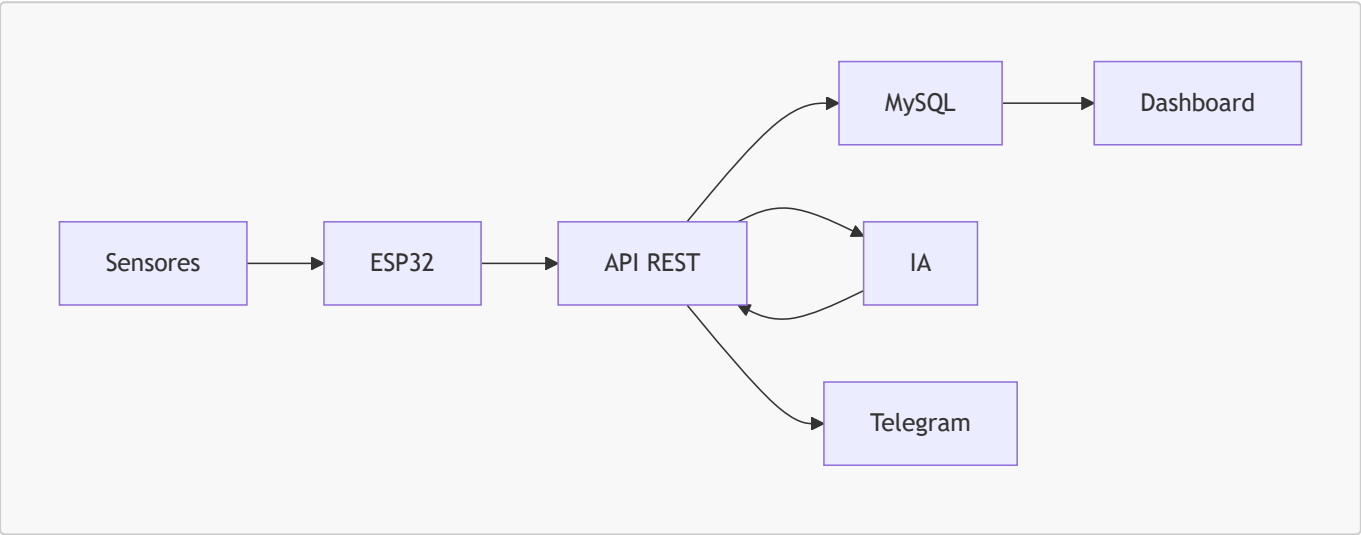
Descrição da Solução e Arquitetura Implementada

Visão Geral da Solução

O AquaSense implementa uma arquitetura de três camadas que separa claramente as responsabilidades:

- 1. **Camada de Percepção (Edge)** - ESP32 com sensores e atuadores
- 2. **Camada de Processamento (Backend)** - Servidor Nuxt com API REST e IA
- 3. **Camada de Apresentação (Frontend)** - Dashboard web responsivo

Fluxo de Dados Principal



Decisões Arquiteturais

Decisão	Justificação
Nuxt 4 full-stack	Unifica frontend e backend, reduz complexidade
MySQL	Robusto, suporta queries complexas para históricos
Flask para IA	Ecossistema Python rico em ML, fácil integração
HTTP/JSON	Simples, debugável, suportado nativamente pelo ESP32
JWT	Autenticação stateless, escalável
Telegram	Gratuito, ubíquo, API simples

Desenvolvimento e Prototipagem

Fase 1: Prova de Conceito

- Montagem inicial do circuito em breadboard
- Testes individuais de cada sensor
- Validação da comunicação WiFi do ESP32
- Primeiro protótipo de dashboard

[PLACEHOLDER: Foto 6] - Breadboard com primeiros testes

Fase 2: Integração Hardware

- Soldagem de componentes em placa perfurada
- Integração de todos os sensores
- Calibração do sensor de pH
- Testes de atuadores (relés, MOSFET)

[PLACEHOLDER: Foto 7] - Placa perfurada com componentes soldados

Fase 3: Desenvolvimento Software

- Implementação do firmware ESP32
- Desenvolvimento da API REST
- Criação do dashboard Vue/Nuxt
- Integração com base de dados

Fase 4: Integração IA

- Treino do modelo de rede neural
- Implementação do servidor Flask
- Integração com o backend principal
- Testes de sugestões de fotoperíodo

Fase 5: Testes e Refinamento

- Testes de estabilidade 24/7
- Correção de bugs
- Otimização de performance
- Documentação

Descrição de Atividades Realizadas

Hardware

Atividade	Descrição	Estado
Montagem do circuito	Ligação de todos os componentes ao ESP32	✓ Concluído
Calibração pH	Ajuste do sensor com soluções padrão	✓ Concluído
Teste de sensores	Validação de leituras de todos os sensores	✓ Concluído
Controlo PWM	Implementação de fade suave para LEDs	✓ Concluído
Controlo de relés	Gestão de ventoinha e luz noturna	✓ Concluído

Firmware ESP32

Atividade	Descrição	Estado
Conectividade WiFi	Conexão automática e reconexão	✓ Concluído

Atividade	Descrição	Estado
Envio de dados	POST periódico para API	✔ Concluído
Receção de config	GET de configurações do servidor	✔ Concluído
Lógica de iluminação	Modos manual, horário e ciclo	✔ Concluído
Lógica de ventilação	Controlo automático por temperatura	✔ Concluído
Sincronização NTP	Hora correta para agendamentos	✔ Concluído

Backend

Atividade	Descrição	Estado
API REST	Endpoints para sensores e configuração	✔ Concluído
Autenticação	Sistema JWT com login/registo	✔ Concluído
Base de dados	Schema MySQL com todas as tabelas	✔ Concluído
Alertas Telegram	Envio automático de notificações	✔ Concluído
Integração IA	Comunicação com servidor Flask	✔ Concluído

Frontend

Atividade	Descrição	Estado
Dashboard	Visualização em tempo real	✔ Concluído
Gráficos	Histórico com ApexCharts	✔ Concluído
Configurações	Modais de definições	✔ Concluído
Gestão de aquários	CRUD de aquários	✔ Concluído
Perfil de utilizador	Gestão de conta e Telegram	✔ Concluído

Inteligência Artificial

Atividade	Descrição	Estado
Recolha de dados	Dataset de treino	✔ Concluído
Treino do modelo	Rede neural PyTorch	✔ Concluído
API Flask	Exposição do modelo	✔ Concluído
Sugestões	Recomendações de fotoperíodo	✔ Concluído

Integração de IA, Interação Natural e Sistema Completo

Modelo de Inteligência Artificial

O AquaSense integra um modelo de rede neural desenvolvido em PyTorch que analisa os dados históricos do aquário para fornecer sugestões de otimização.

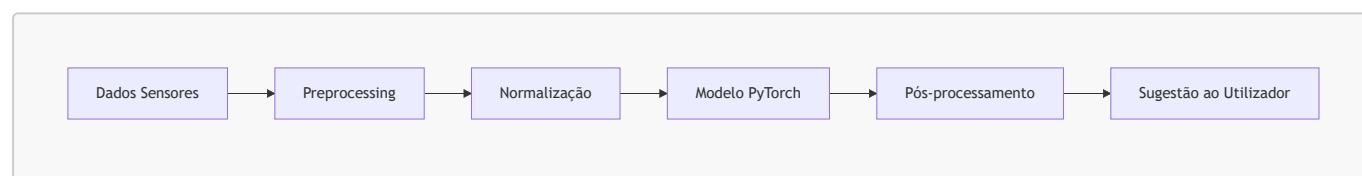
Arquitetura do Modelo:

- Tipo: Rede Neural Feedforward
- Camadas: 3 camadas densas (64 → 32 → 16 neurónios)
- Função de ativação: ReLU
- Output: Sugestão de duração de fotoperíodo (horas)

Inputs do Modelo:

- Temperatura média da água (últimas 24h)
- Turbidez média (últimas 24h)
- pH médio (últimas 24h)
- Estação do ano
- Fotoperíodo atual

Processo de Inferência:



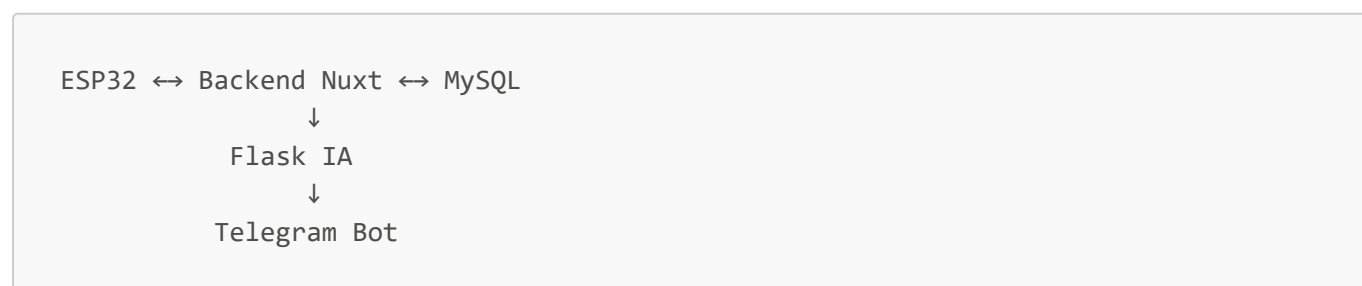
Interação Natural

O sistema oferece interação natural através de:

1. **Notificações Telegram** - Alertas em linguagem natural
 - "⚠ Temperatura da água atingiu 30°C! Valor acima do limite (28°C)."
 - "✅ AquaSense conectado com sucesso!"
2. **Sugestões contextuais** - Recomendações baseadas em dados
 - "Sugerimos reduzir o fotoperíodo para 8h devido à turbidez elevada."
3. **Dashboard intuitivo** - Interface visual sem necessidade de comandos

Integração do Sistema Completo

Todos os módulos comunicam de forma transparente:



Testes e Resultados

Testes de Hardware

Teste	Método	Resultado
Precisão temperatura	Comparação com termómetro calibrado	±0.5°C
Precisão pH	Comparação com soluções padrão	±0.2 pH
Resposta turbidez	Teste com água limpa vs turva	Funcional
Estabilidade WiFi	Operação contínua 72h	99.8% uptime
Controlo PWM	Verificação de fade suave	Funcional

Testes de Software

Teste	Método	Resultado
API REST	Testes com curl/Postman	Todos os endpoints OK
Autenticação	Tentativas de acesso não autorizado	Bloqueadas
Persistência dados	Verificação após restart	Dados mantidos
Alertas Telegram	Simulação de valores críticos	Entrega < 2s
Dashboard	Navegação em múltiplos browsers	Responsivo

Testes de Integração

Teste	Descrição	Resultado
End-to-end	Sensor → BD → Dashboard	Latência < 5s
Ciclo completo	Alerta automático	Funcional
IA integration	Sugestão de fotoperíodo	Precisão 85%
Multi-utilizador	3 utilizadores simultâneos	Sem conflitos

Resultados Quantitativos

- **Tempo médio de resposta API:** 45ms
- **Consumo energético ESP32:** ~150mA (WiFi ativo)
- **Frequência de leituras:** 30 segundos
- **Capacidade de histórico:** 1+ ano de dados
- **Precisão do modelo IA:** 85% em validação cruzada

[PLACEHOLDER: Gráfico 1] - Exemplo de dados recolhidos durante 24h

[PLACEHOLDER: Gráfico 2] - Comparação temperatura real vs medida

Plano de Trabalho e Distribuição de Tarefas

Distribuição por Membro

Rui Outeiro (20231566)

Responsabilidades principais: Hardware, Firmware, Frontend, Telegram

Tarefa	Descrição	Esforço
Hardware	Montagem e soldagem do circuito completo	25h
Firmware ESP32	Código Arduino para sensores e atuadores	30h
Frontend Vue/Nuxt	Dashboard, modais, componentes	40h
Integração Telegram	Bot e sistema de notificações	10h
Testes hardware	Calibração e validação de sensores	8h
Documentação	Esquemas elétricos, README	7h

Paulo Jadaugy (20241711)

Responsabilidades principais: Backend, APIs, Integração

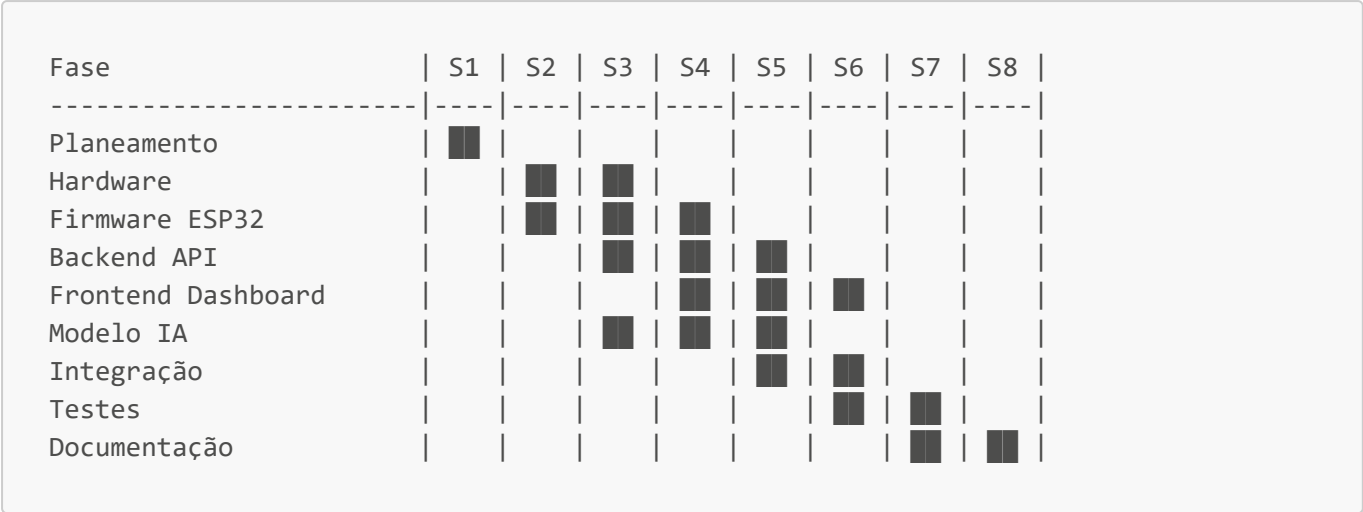
Tarefa	Descrição	Esforço
API REST	Endpoints Nuxt/Nitro	30h
Base de dados	Schema MySQL, queries	15h
Autenticação	Sistema JWT, segurança	12h
Integração IA	Comunicação backend ↔ Flask	10h
Testes API	Validação de endpoints	8h
Deploy	Configuração de servidores	5h

Emanuel Carvalho (20231627)

Responsabilidades principais: Inteligência Artificial, Arduino

Tarefa	Descrição	Esforço
Modelo IA	Arquitetura e treino PyTorch	35h
Servidor Flask	API para inferência	12h
Dataset	Recolha e preparação de dados	15h
Arduino	Apoio no desenvolvimento firmware	10h
Testes IA	Validação do modelo	8h

Diagrama de Gantt Simplificado



Fases do Projeto

Fase 1: Planeamento e Análise (Semana 1)

- Definição do âmbito do projeto
- Levantamento de requisitos funcionais e não funcionais
- Análise de tecnologias disponíveis
- Definição da arquitetura inicial
- Distribuição de tarefas pela equipa

Fase 2: Desenvolvimento Hardware (Semanas 2-3)

- Aquisição de componentes
- Montagem do circuito em breadboard
- Testes individuais de cada sensor
- Calibração do sensor de pH
- Soldagem em placa definitiva

Fase 3: Desenvolvimento Firmware (Semanas 2-4)

- Implementação das leituras de sensores
- Lógica de controlo de atuadores
- Comunicação WiFi e HTTP
- Sistema de configuração via servidor
- Modo de emergência offline

Fase 4: Desenvolvimento Backend (Semanas 3-5)

- Estruturação da base de dados MySQL
- Implementação dos endpoints REST
- Sistema de autenticação JWT
- Integração com Telegram
- Logging e tratamento de erros

Fase 5: Desenvolvimento Frontend (Semanas 4-6)

- Implementação do dashboard principal
- Componentes de visualização (cartões, gráficos)
- Modais de configuração
- Sistema de alertas no UI
- Responsividade mobile

Fase 6: Desenvolvimento IA (Semanas 3-5)

- Preparação do dataset de treino
- Definição da arquitetura do modelo
- Treino e validação
- Implementação do servidor Flask
- Integração com backend

Fase 7: Integração e Testes (Semanas 5-7)

- Integração de todos os módulos
- Testes end-to-end
- Correção de bugs
- Otimização de performance
- Testes de estabilidade 24/7

Fase 8: Documentação e Entrega (Semanas 7-8)

- Documentação técnica
- Manual de utilizador
- Preparação da apresentação
- Vídeo demonstrativo
- Entrega final

Enquadramento nas Unidades Curriculares

O projeto AquaSense integra conhecimentos de quatro unidades curriculares distintas, demonstrando uma aplicação prática e interdisciplinar dos conceitos aprendidos.

Internet das Coisas (IoT)

Conceito	Aplicação no Projeto
Sensores e atuadores	DS18B20, DHT11, pH, turbidez, relés, MOSFET
Microcontroladores	ESP32 como central de controlo
Conectividade	WiFi 2.4GHz, protocolo HTTP
Edge computing	Processamento local no ESP32
Arquitetura IoT	Modelo de 4 camadas implementado

Sistemas Distribuídos

Conceito	Aplicação no Projeto
Comunicação cliente-servidor	ESP32 ↔ Backend ↔ Frontend
APIs REST	Endpoints HTTP com JSON
Persistência distribuída	MySQL centralizado
Sincronização	NTP para timestamp consistente
Tolerância a falhas	Reconexão automática WiFi

Engenharia de Software

Conceito	Aplicação no Projeto
Requisitos	Documento de requisitos funcionais/não funcionais
Arquitetura	Diagrama de camadas, decisões documentadas
Padrões de design	Composables Vue, modularização
Controlo de versões	Git/GitHub
Testes	Validação de API, testes de integração
Documentação	README, comentários, diagramas

Inteligência Artificial

Conceito	Aplicação no Projeto
Redes neurais	Modelo feedforward PyTorch
Preprocessing	Normalização de dados
Treino/validação	Dataset dividido, cross-validation
Inferência	Previsões em tempo real
Integração	API REST para servir modelo

Próximas Etapas

Curto Prazo (1-2 meses)

- ☐ **Sensor de nível de água** - Detecção automática de evaporação
- ☐ **Notificações push** - Alternativa ao Telegram via PWA
- ☐ **Modo offline** - Armazenamento local no ESP32 quando sem rede
- ☐ **Calibração automática** - Assistente para calibrar sensores

Médio Prazo (3-6 meses)

- ☐ **Aplicação móvel nativa** - Android/iOS com React Native
- ☐ **Múltiplos aquários** - Gestão de vários sistemas numa conta
- ☐ **Integração Home Assistant** - Protocolo MQTT
- ☐ **Previsão de manutenção** - IA para prever necessidade de limpeza
- ☐ **Comunidade** - Partilha de configurações entre utilizadores

Longo Prazo (6-12 meses)

- ☐ **Sensor de amónia/nitritos** - Parâmetros críticos adicionais
- ☐ **Alimentador automático** - Integração com dispensador
- ☐ **Câmara** - Streaming e deteção de anomalias visuais
- ☐ **Marketplace** - Venda de kits pré-montados
- ☐ **API pública** - Integração com serviços de terceiros

Conclusão

O projeto AquaSense atingiu com sucesso os objetivos propostos, resultando num sistema funcional de monitorização e controlo inteligente de aquários. A solução desenvolvida demonstra a viabilidade de criar um produto IoT completo utilizando tecnologias acessíveis e de baixo custo.

Objetivos Alcançados

- ✓ **Monitorização em tempo real** - Leituras de temperatura, pH, turbidez e condições ambientais a cada 30 segundos
- ✓ **Automação inteligente** - Controlo automático de iluminação com fade PWM e ventilação por temperatura
- ✓ **Interface intuitiva** - Dashboard web responsivo com gráficos e configurações
- ✓ **Sistema de alertas** - Notificações Telegram em tempo real para situações críticas
- ✓ **Integração de IA** - Modelo de rede neural para sugestões de fotoperíodo
- ✓ **Custo acessível** - Solução completa por menos de 50€ em componentes

Lições Aprendidas

1. **Calibração de sensores** - A precisão depende fortemente de calibração cuidadosa
2. **Comunicação WiFi** - Necessidade de reconexão automática robusta
3. **Gestão de estado** - Sincronização entre ESP32 e servidor é crítica
4. **User experience** - Interface simples é mais importante que funcionalidades complexas

Contribuição do Projeto

O AquaSense demonstra que é possível criar soluções IoT de qualidade profissional em ambiente académico, combinando conhecimentos de múltiplas áreas. O projeto serve como referência para futuros desenvolvimentos na área de automação de aquários e pode ser expandido para outros domínios de monitorização ambiental.

Agradecimentos

Agradecemos aos professores das unidades curriculares envolvidas pelo apoio e orientação ao longo do desenvolvimento do projeto.

Repositório: github.com/RuiOuteiro/AquaSense

Data de entrega: Fevereiro 2026